Technical University of Crete
School of Electrical and Computer Engineering
Course: Optimization
Exercise 4
Delivery Date: January 16, 2024
Instructor: Athanasios P. Liavas
Points: 100/1000

Paraskevopoulou Despoina
2021030135

In this exercise we shall study the important topic of Linear Programming. We have the following problem:

$$\text{minimize} \;\; f(x) = c^T x$$

$$\text{subject to} \;\; \mathbf{Ax} = \mathbf{b}$$

$$\mathbf{x} \geq \mathbf{0}$$

where $A \in \mathbb{R}^{p \times n}$ with $\mathbf{rank}(\mathbf{A}) = p$ and $\mathbf{b} \in \mathbb{R}^p$.
First of all, we started by generating random $\mathbf{A}$, a random nonnegative $\mathbf{x}$ and we computed the $\mathbf{Ab} = \mathbf{x}$.

1. At the beginning we started by solving the problem via **cvx**. The solution that we got from that is given below.

```
Status: Solved
Optimal value (cvx_optval): -4.02078

Optimal value of x:
    0.0000
    5.3260

Optimal value of the objective function:
   -4.0208
```

Figure 1: The cvx solution

2. Then, we will proceed to solve the problem using the interior point method with logarithmic barrier, starting from a feasible point that we get using the cvx.
First, we define the function ϕ as follows:

$$\phi(x) = -\sum_{i=1}^{m} log(-f(x))$$

From implementing the interior point method we got the following plot. We plotted the feasible point (blue star), the feasible set (cyan), the trajectories are depicted with red bullets and the optimal solution is the red star.

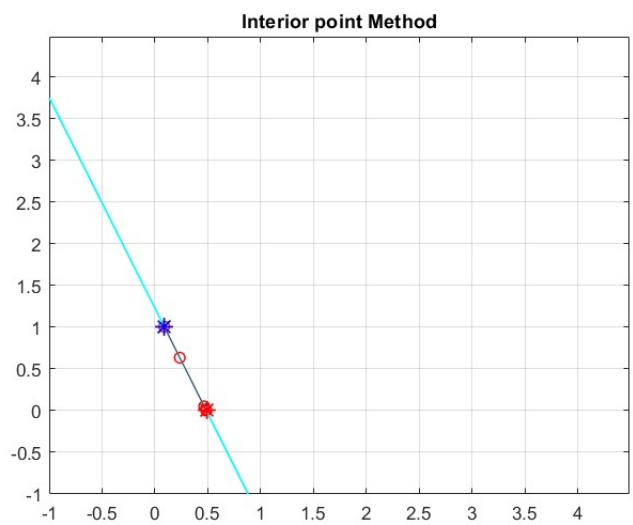

Figure 2: Interior point Method

Here is the same plot but in zoom so we can see better the trajectories:

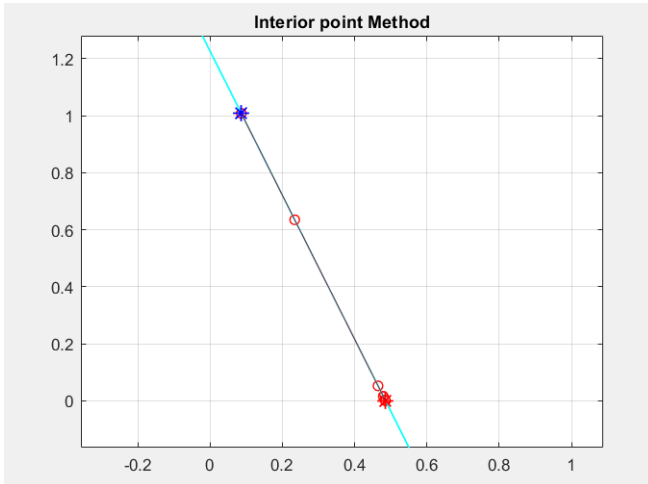

Figure 3: Interior point Method in zoom

3. Afterwards, we solved the initial problem using the primal-dual algorithm. For that we start from a point near the feasible point so it satisfies all the inequality constraints. In order to implement the algorithm we have to define, firstly, the vector $r_t(x, \lambda, v) = \begin{bmatrix} \nabla f_0(x) + Df(x)^T\lambda + \mathbf{A}^T\mathbf{v} \\ -diag(\lambda)f(x) - \frac{1}{t}1 \\ \mathbf{Ax} - \mathbf{b} \end{bmatrix} = \begin{bmatrix} c - \lambda + \mathbf{A}^T\mathbf{v} \\ -diag(\lambda)x - \frac{1}{t}\mathbf{1} \\ \mathbf{Ax} - \mathbf{b} \end{bmatrix}$.
From the vector we define as **dual residual** the $r_{t,d} = c - \lambda + \mathbf{A}^T\mathbf{v}$, **primal residual** the $r_{t,p} = -diag(\lambda)x - \frac{1}{t}\mathbf{1}$ and **centrality residual** the $r_{t,c} = \mathbf{Ax} - \mathbf{b}$. The search direction is defined as $\delta y_{pd} = (\Delta x_{pd}, \Delta\lambda_{pd}, \Delta v_{pd})$ and we should solve the linear system:

$$\begin{bmatrix} O_{n\times n} & \mathbb{I}_{n\times n} & \mathbf{A}^T \\ diag(\lambda) & diag(\mathbf{x}) & O_{n\times p} \\ \mathbf{A} & O_{p\times n} & O_{p\times p} \end{bmatrix} \begin{bmatrix} \Delta x_{pd} \\ \Delta\lambda_{pd} \\ \Delta v_{pd} \end{bmatrix} = - \begin{bmatrix} r_{t,d}(y) \\ r_{t,c}(y) \\ r_{t,p}(y) \end{bmatrix}$$

We got the following results:

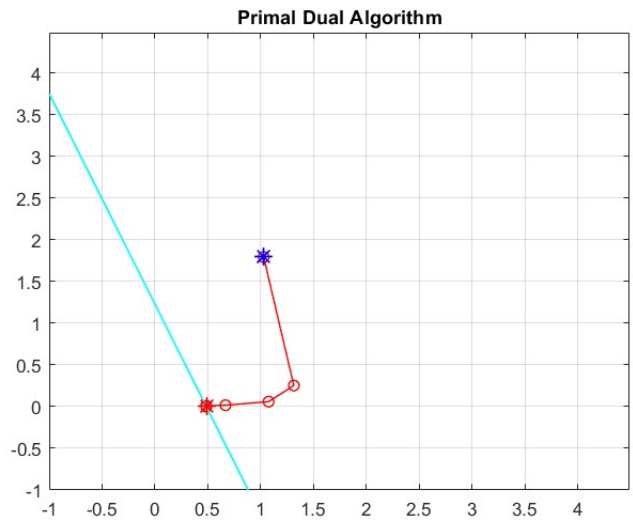

Figure 4: Primal Dual Algorithm

Here is the same plot in zoom :

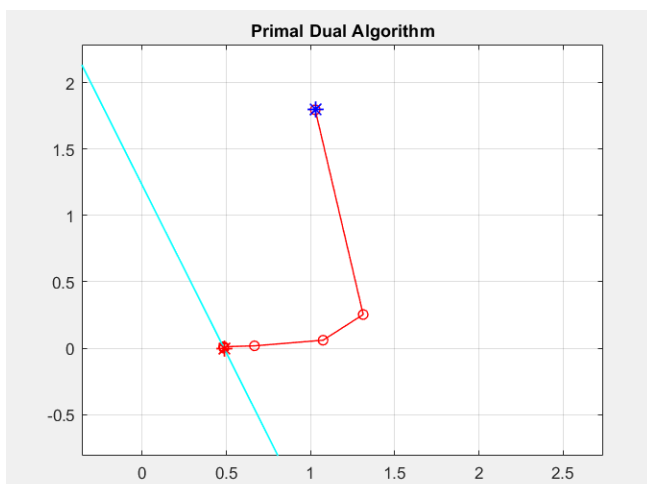

Figure 5: Primal Dual Algorithm in zoom

We, also, counted the nonzero elements in the optimal solution and we compared with p, which is the rank of the matrix $\mathbf{A}$. So, we observed that the nonzero elements in the solution were always equal to the rank of matrix $\mathbf{A}$. We conclude that the number of nonzero elements in the optimal solution is directly related to the number of linearly independent rows in $\mathbf{A}$ and we understand the the equality constraints are

deeply associated with matrix **A**.
Additionally, we observe that the interior point method does more iterations in order to reach the optimal solution as we see in the screenshot below:

```
Interior Point Method
Total iterations: 25 | f = -0.502852
****************************************

Primal Dual Algorithm
Total iterations: 13 | f = -0.502852
****************************************
```

Figure 6: Iterations and value of the optimal point

This leads us to believe that the primal dual algorithm is more efficient.
After running a few experiments for the algorithms we observed, among others, the following things:

(1) The interior point method, decreases the number of outer iterations when we increase the value of μ.

(2) By increasing μ, the number of inner iterations are increasing (we checked that with a simple counter in the code).

Those two observations lead us to the understanding that there is a trade-off that comes with the choice of the constant μ.

The code for the interior point method:
Matlab Code:

```
function [x, fun_value, rec] =
interiorPointMethod(A, b, c, n, x0,x_opt, t1, t2, alpha,
   beta)
% Initialization
threshold_1 = t1;
threshold_2 = t2;
h = 1;
mu = 10;
% Linear constraint parameters: a_1^T x -b_1 <= 0
a_1 = A;
b_1 = b;
% Parameter of linear Cost Function f(x) = c^T x
x_init(:,1) = x0;
t = 1;
fun_value = f(c,x_init(:,1));
f_rec = fun_value;
outer_iter = 1;
X(:,outer_iter) = x_init;
k_rec = outer_iter;
x_rec = x0;

l_x = 1000;
while (1)

     x(:,1) = X(:,outer_iter);
     inner_iter = 1;
     while ( 1 )
         grad = h*gradf(x_init,c,t,n);
         Hess = h*hessf(x_init,n);
         w = -inv(A*inv(Hess)*A')*A*h*inv(Hess)*grad;
         Dx_Nt = -inv(Hess)*(grad+A'*w);          % Newton
             step
         l_x_ = l_x;
         l_x = sqrt(Dx_Nt'*Hess*Dx_Nt);
         if (l_x^2/2 <= threshold_1 || abs(l_x - l_x_)
            <=10^-7) break; end
```

```

        tau = 1;
        x_new = x(:,inner_iter) + tau * Dx_Nt;
        while ( not(all(x_new > 0)) )
            tau = beta * tau;
            x_new = x(:,inner_iter) + tau * Dx_Nt;
        end
        % !!!! x_new is FEASIBLE here !!!
        % Backtracking
        while ( t*f(c, x_new) + logBarrier(x_new) >
        t*f(c, x(:,inner_iter)) + logBarrier(x(:,
           inner_iter)) + alpha*tau*grad'*Dx_Nt)
             tau = beta * tau;
             x_new = x(:,inner_iter) + tau * Dx_Nt;
        end

        % Update x
        x(:,inner_iter+1) = x(:,inner_iter) + tau * Dx_Nt
           ;    % Update x
        inner_iter = inner_iter + 1;
    end

    X(:,outer_iter+1) = x(:,inner_iter);
    if (1/t < threshold_2)
        break; end

    if(size(x(:,inner_iter),1) == 2)
       plot(x_rec(1,:),x_rec(2,:),'-ro',LineWidth=0.85)
       hold on
       grid on;
    end

    outer_iter = outer_iter + 1;
    t = t * mu;
    fun_value = f(c,x(:,inner_iter));
    x_rec = [x_rec, x(:,inner_iter)];
    k_rec = [k_rec, outer_iter];
    f_rec = [f_rec, fun_value];
end
```

```

fprintf("Total iterations: %d | f = %f\n", outer_iter, fun_value);
rec = [k_rec; f_rec];

x_ = -2:0.1:max(x_opt)+5;
y_ = -(A(1)/A(2))*x_ + b/A(2);
plot(x_, y_, 'c', LineWidth=1)
axis([-1 max(x_opt)+4 -1 max(x_opt)+4])
hold on;

plot(x0(1), x0(2), 'b*', 'MarkerSize', 10, 'LineWidth', 1);
hold on;
plot(x_opt(1), x_opt(2), 'r*', 'MarkerSize', 10, 'LineWidth', 1);
hold on;
plot(X(1,:), X(2,:))
title('Interior point Method');
grid on;
hold off;
```

Here, we present the code used for the implementation of the primal-dual algorithm:
Matlab Code:

```
%% Primal Dual Algorithm
function [x, fun_value, rec] =
primalDual(A, b, c, n, x0, x_opt, t1, t2, alpha, beta, m,
   mu)
[p, n] = size(A);
x = x0;
t = 1;
lambda = (1/t)*ones(n,1)./x;
v = A'\(lambda-c);

y = [x; lambda; v];
k = 0;
h = x'*lambda;
x_rec = x0;
f_rec = f(c,x);
k_rec = k;
figure;
while(1)

    t = mu*m/h;

    F = [zeros(n,n), -eye(n), A'; diag(lambda),
    diag(x), zeros(n,p); A, zeros(p,n), zeros(p,p)];

    delta_y = -F\r(A, b, c, t, y);
    delta_x = delta_y(1:n);
    delta_l = delta_y(n+1:2*n);
    delta_v = delta_y(2*n+1:2*n+p);

    temp = 1;
    for i = 1:n
        if(-lambda(i)/delta_l(i)<temp && delta_l(i) < 0)
            temp = -lambda(i)/delta_l(i);
        end
    end

```

```
    s_max = min(1,temp);

    s = 0.99*s_max;

    x_new = x + s*delta_x;

    while(not(all(x_new > -10^-15)))
        %disp('88');
        s = beta*s;
        x_new = x + s*delta_x;
    end

    lambda_new = lambda + s*delta_l;
    v_new = v + s*delta_v;
    y_new = [x_new; lambda_new; v_new];

    while(norm(r(A, b, c, t, y_new)) > (1-alpha*s)*norm(r(
       A, b, c, t, y)))
        %disp('99');
        s = beta*s;
        x_new = x + s*delta_x;
        lambda_new = lambda + s*delta_l;
        v_new = v + s*delta_v;
        y_new = [x_new; lambda_new; v_new];
    end

    x = x + s*delta_x;
    lambda = lambda + s*delta_l;
    v = v + s*delta_v;
    y = [x; lambda; v];
    k = k + 1;
    fun_value = f(c,x);
    x_rec = [x_rec, x];
    k_rec = [k_rec, k];
    f_rec = [f_rec, fun_value];

    if(n == 2)
        plot(x_rec(1,:),x_rec(2,:),'-ro',LineWidth=0.85)
        hold on
```

```
        grid on;
    end

    h = x'*lambda;
    residual = r(A, b, c, t, y);
    if((norm(residual(2*n+1:2*n+p))<=t1) && norm(residual(1:n))<=t1 && (h < t2))
        break
    end
    if(k == 1000) break; end
end
fprintf("Total iterations: %d | f = %f\n", k, fun_value);
rec = [k_rec; f_rec];

x_ = -2:0.1:max(x_opt)+5;
y_ = -(A(1)/A(2))*x_ + b/A(2);
plot(x_, y_, 'c', LineWidth=1)
axis([-1 max(x_opt)+4 -1 max(x_opt)+4])
hold on;

plot(x0(1), x0(2), 'b*', 'MarkerSize', 10, 'LineWidth', 1);
hold on;
plot(x_opt(1), x_opt(2), 'r*', 'MarkerSize', 10, 'LineWidth', 1);
title('Primal Dual Algorithm');
grid on;
hold off;

end
```